Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

a. Team name: **Localhost**
b. Team leader name: **Tejas Nafde**
c. Problem Statement: **Future of Work & Enterprise Productivity**

Brief about the idea

Canon

Canon is the shared memory of a company, for its people and its AI agents.

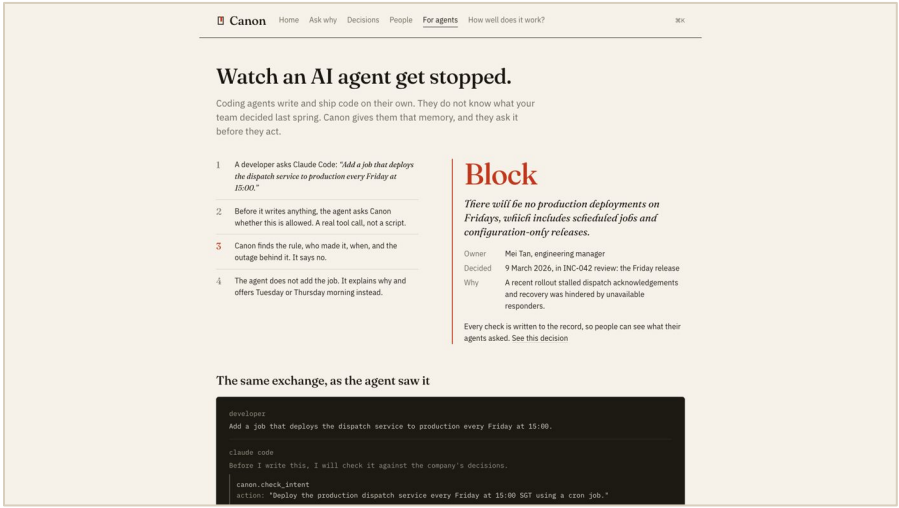
Teams decide things in Slack threads, meetings and pull requests, then forget. Wiki pages go out of date. New hires and coding agents act on old rules.

Canon keeps one record of decisions: rationale, owner, date, authority, and a quote checked word for word against its source.

It checks every new message, document and agent action against that record, and warns when one is about to break a decision.

Coding agents connect over MCP and call check_intent before they act. A Friday deploy job is blocked, with the incident review as evidence.

Live on a public Discord server: post a message, and Canon replies within a minute.



canon.tn07.dev: an AI agent gets stopped

Opportunities

How is it different?	How it solves the problem	USP: agent-native
- Meeting-memory tools summarise. Decision logs store. Spec checkers review code. - Canon does all three on one record, and knows who may change a rule: official, proposed or claim, from the org tree. - Its users include AI agents, not only people.	- Extracts decisions with verified quotes from chats, meetings, wiki, PRs, PDFs, screenshots and voice recordings. - Reconciles each one: same, update or contradiction, ordered by effective date. - Flags out-of-date documents and broken premises, and sends unverified claims to the right lead. - Answers why with citations, or says I do not know.	- An MCP server any agent can call: check_intent returns allow, warn or block with the deciding decision. - Every agent read, write and check is recorded for people to see. - Also an ADK agent over A2A, and a Discord bot.

List of features offered by the solution

Ingest anything: Slack, meetings, wiki, PRs, commits, email, paste, PDFs, screenshots, audio and video (Gemini reads them; 10 min audio, 2 min video, 30 min a day).

Decision extraction: statement, rationale, assumptions, owner, date, quotes verified word for word.

Reconciliation: same, update (supersedes) or contradiction, ordered by effective date. Older copies marked outdated.

Authority: official, proposed or claim from the org tree. Unverified claims go to the right lead. Jokes are ignored.

Drift alerts: out-of-date documents and broken premises, both quotes side by side.

Ask why: cited answer and timeline; refuses when the record is silent.

I am new here: five things you must know, a page not to trust, people to ask.

Catch me up: 14 roles, a summary and a ranked to-do list with reasons.

Agent guard over MCP: check_intent gives allow, warn or block. Six tools for Claude Code and Codex. ADK agent over A2A.

Discord, live: synced every minute (threads, files, PDFs, screenshots, voice notes); replies under the message; /canon ask, check, catchup, remember.

People and org: who owns what, who left, who took over.

Measured: gold-set benchmark and a memory-health replay, served in the app.

Cost controls: daily Gemini cap, Discord caps, scale to zero.

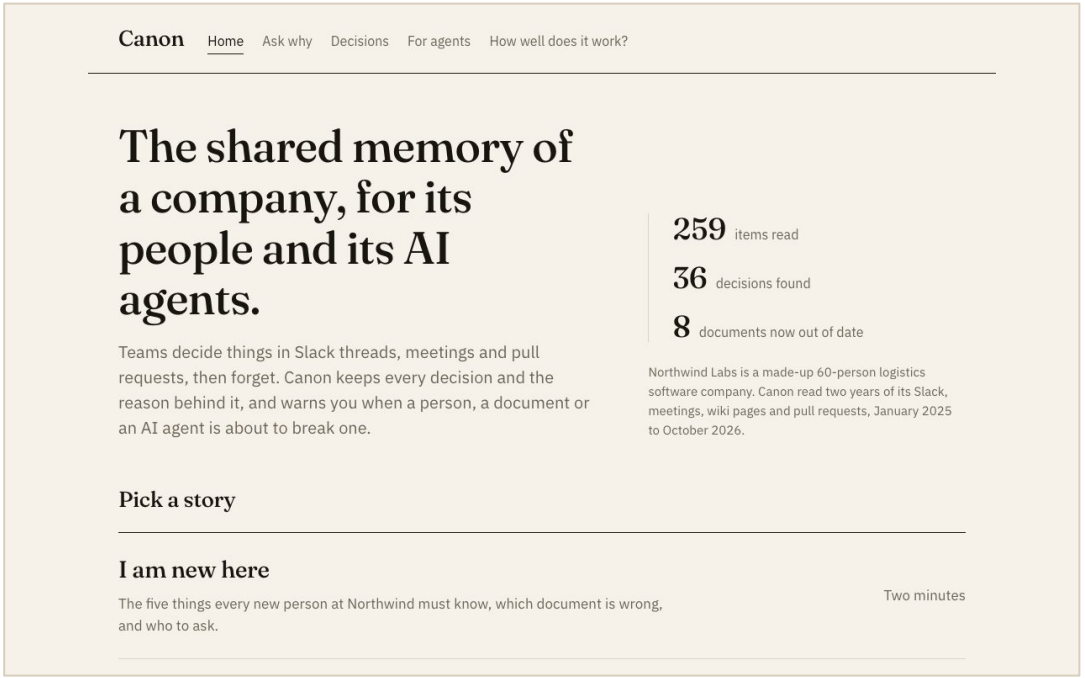
Process flow diagram

A. A new source arrives (web paste, Discord, PR, agent, PDF)

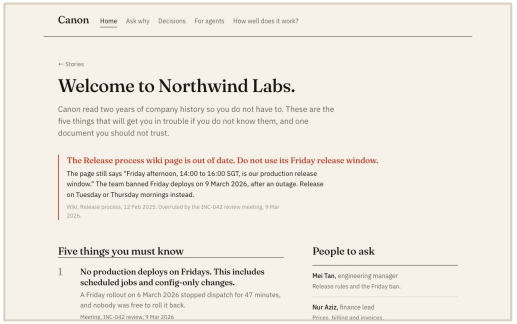


B. A person or an agent asks or acts

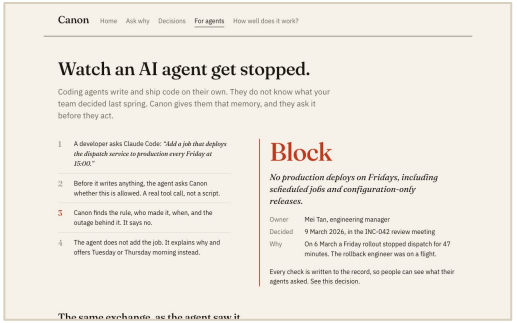
Wireframes / mock diagrams



Home: stories and live counts

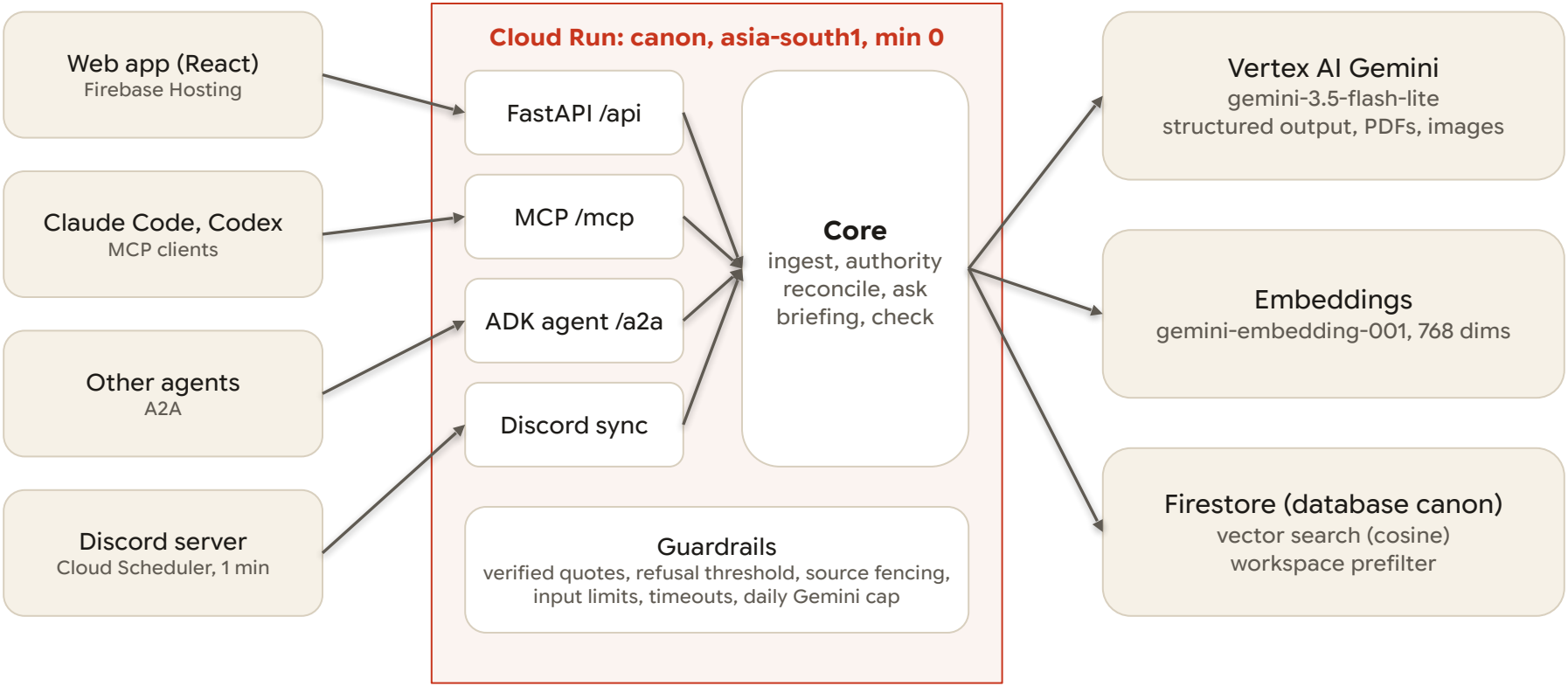


I am new here



For agents: the block

Architecture diagram



Technologies used in the solution

Technology	What Canon uses it for
Gemini 3.5 Flash-Lite on Vertex AI	Extraction, reconciliation, drift checks, answers, briefings, verdicts; structured output; reads PDFs and images
gemini-embedding-001 (768 dims)	Embeddings for source chunks, decisions and queries
Cloud Firestore, named database canon	Sources, chunks, decisions, alerts, activity, people; vector search with a workspace prefilter
Cloud Run	One scale-to-zero service: API, MCP server, A2A agent, Discord sync and slash commands
Firebase Hosting	Static web app; rewrites /api, /mcp, /a2a, /hooks, /discord to Cloud Run on one origin
Cloud Scheduler	Calls the Discord sync every minute, so no always-on gateway is needed
Secret Manager, Cloud Monitoring	One secret for every Canon credential; alert policies relayed to Discord
ADK + A2A, MCP (FastMCP)	Canon as an LlmAgent with four tools; six MCP tools over streamable HTTP
Python 3.12, FastAPI, uv, pytest	Backend; 85 tests replay recorded Gemini fixtures, no network
React, TypeScript, Vite, Tailwind	Editorial web app: stories, catch-up, agents, paste, ask, decisions, people, benchmark

Estimated implementation cost

Idle cost is near zero. Cloud Run scales to zero (min instances 0). No per-hour resource.

Firestore: the demo data fits inside the free tier.

Gemini is paid per call. An answer is one embedding and one Flash-Lite call. A check is one call, or none when no decision is relevant. An ingest is one extraction plus a few reconcile calls.

Refusals and irrelevant checks never reach Gemini. The similarity threshold stops them first.

Tracked, not guessed: Gemini calls, Cloud Run, Artifact Registry and Hosting carry the label app=canon, so the billing export splits Canon's spend.

What we measured:

- Reading the whole 380-source archive (a full seed) costs about INR 80 to 100 in Gemini calls.

- Live checks and answers: one call each, about 1 to 2 seconds.

Hard caps:

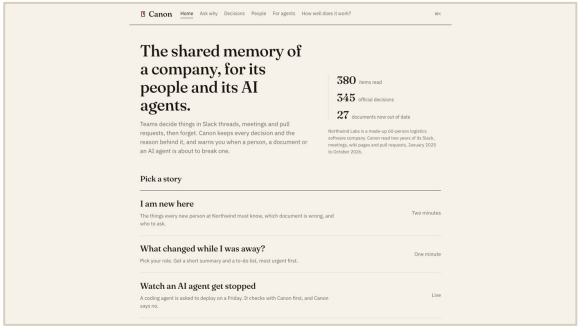
- Daily Gemini cap per instance (1500 generate calls by default).

- Discord: 200 messages a day, 20 per user; slash commands stop at the cap.

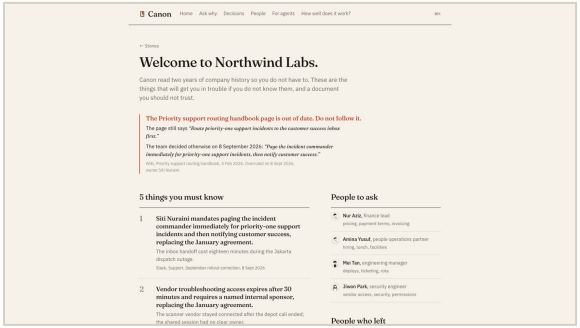
- 120 POST requests per minute per IP.

Not measured yet: tokens per call at scale, so we do not quote a per-user price.

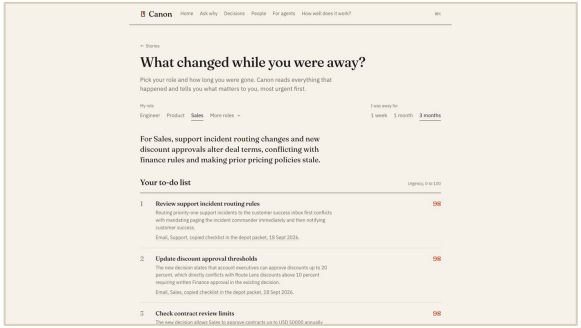
Snapshots of the prototype



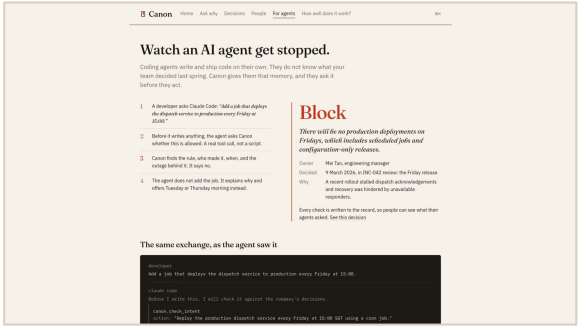
Home: four stories and live counts



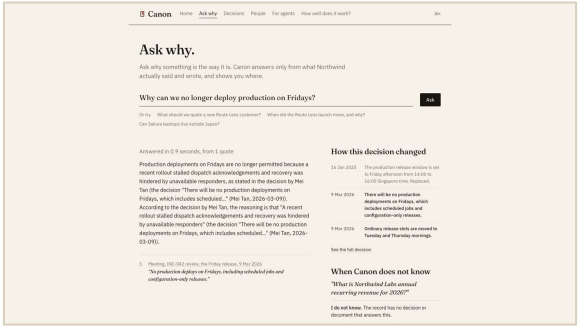
I am new here: out-of-date page, who to ask



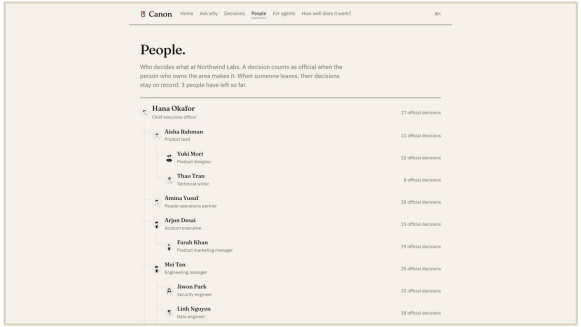
Catch me up: sales, ranked to-do list



An AI agent gets stopped: Block



Ask why: cited answer and timeline



People: org tree, who left, successors

Prototype performance report / benchmarking

Task (labelled gold set)	Planted	Found	Score
Decisions extracted	82	82	100%
Contradictions and updates	39	35	90%
False positive alerts		2	
Out-of-date documents	19	18	95%
check_intent verdicts	37	32	86%
Unanswerable, refused	5	5	100%
Latency p50 / p95 (ask, check)		1221 / 1795 ms	

Run 2026-10-17 19:10 UTC, model gemini-3.5-flash-lite, dataset acme4: 380 synthetic sources (about 100 pure noise), 14 roles. check_intent re-measured 2026-10-18.

Reproduce: backend/scripts/seed.py, then eval.py. The app serves the report at /api/eval.

Memory health: replay in time order

Check	Result
Decisions with no traceable source	0 of 10
Duplicate decisions	0
Quarter checkpoints correct	3 of 3
Supersession chains correct	1 of 1
Authority cases correct	12 of 12

In the open:

- premise changes found 2/5, our weakest part
- updates 19/20, contradictions 16/19
- answerable questions 21/21 cited an expected decision
- 579 stored quotes, all verified word for word
- check misses lean to caution (warn)

Additional details / future development

Where people already talk: Slack and Microsoft Teams next to the live Discord connector, answering in place.

Permissions: sign-in, per-team visibility, and decisions that follow the access rules of their source.

Code review at scale: check PR diffs against the record and post review comments for every repo.

Beyond software: manufacturing SOP drift. A line change or a supplier note that contradicts the approved procedure raises an alert before a shift runs it.

Better recall: premise watch finds 2 of 5 today; we will tune it against the gold set.

Real data: this repository's own README, PRs and commits next to the synthetic corpus.

Links

GitHub public repository: https://github.com/tejasnafde/canon

Demo video: (add the video link)

Final product link: https://canon-tn07.web.app (also https://canon.tn07.dev)

Cloud Run: https://canon-ezvux7gqxa-el.a.run.app/health

Live Discord demo: https://discord.gg/HwSgjmcgF5

Connect your agent: claude mcp add --transport http canon https://canon.tn07.dev/mcp

No login needed. All company data is synthetic and labelled as such.

Canon

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

